



برنامه نویسی شی گرا

Object Oriented Programming

مدرس:

مریم خراشادیزاده

اشاره گر this

- همانطور که قبلا هم گفتیم هر شی، نسخه خود را از داده‌های کلاس دارد؛ اما **تنها یک نسخه** از توابع کلاس وجود دارد.
- با توجه به نکته فوق یک سوال پیش می‌آید که چگونه مشخص می‌شود که تابع عضو کلاس روی داده کدام شی، عملیات خود را انجام دهد؟ **برای پاسخ به این سوال باید با کلمه کلیدی جدیدی در C++ به نام this آشنا شویم.**
- **this** اشاره‌گری است که در هر شی، برای دسترسی به آدرس خودش تعریف شده است.
- اشاره گر **this**، به عنوان بخشی از شی در نظر گرفته نمی‌شود و حافظه مربوط به آن در اندازه شی لحاظ نمی‌شود؛ اما می‌توان در توابع عضو کلاس از آن برای **دسترسی به داده‌های کلاس** استفاده کرد.
- اشاره گر **this** برای مشخص شدن شئی که تابع کلاس با استفاده از آن فراخوانی می‌شود، **به طور ضمنی** (توسط کامپایلر) به **توابع عضو غیرایستای** کلاس ارسال می‌شود.
- اشاره گر **this** به **توابع عضو ایستای** کلاس به طور ضمنی ارسال نمی‌شود. **چرا؟**
- اشاره گر **this** به صورت ضمنی - همین‌طوری که تا کنون بدون آوردن کلمه کلیدی **this** از آن در دسترسی به داده‌های شی از آن استفاده کردیم. - یا صریحا در توابع عضو غیرایستا مورد استفاده قرار می‌گیرد.



مثال ۱: اشاره گر this (فایل Point.h)

- نکته ۱: استفاده از اشاره گر this در تابع Setx و Sety مشکل تداخل نام بین متغیر کلاس و پارامتر ورودی (و یا سایر متغیرهای محلی تابع) را حل می کند. پس می توان متغیر کلاس و پارامتر ورودی و یا متغیر محلی تابع را همنام انتخاب کرد.
- نکته ۲: استفاده از اشاره گر this در تابع Getx و Gety اختیاری است و می توان مشابه قبل تنها از x و y استفاده کرد.

استفاده از اشاره گر this

```
1  #ifndef POINT_H
2  #define POINT_H
3
4
5  class Point
6  {
7      friend void func(Point&);
8      public:
9          Point(int = 0, int = 0);
10         ~Point();
11
12         int Getx() const { return this -> x; }
13         void Setx(int x) { this -> x = x; }
14         int Gety() const { return this -> y; }
15         void Sety(int y) { this -> y = y; }
16         void display() const;
17         static unsigned int GetCount();
18
19     protected:
20
21     private:
22         int x;
23         int y;
24         static unsigned int count;
25 };
26
27 #endif // POINT_H
28
```



مثال ۱: اشاره گر this (فایل Point.cpp)

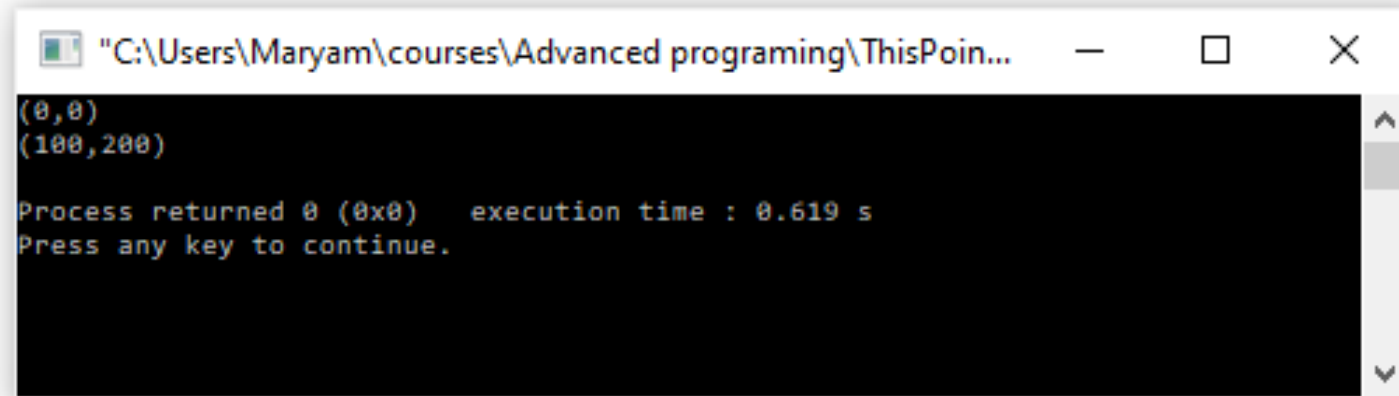
- نکته ۱: مشابه اسلاید قبل می توان از اشاره گر this در توابع غیرایستا استفاده کرد.
- نکته ۲: استفاده از اشاره گر this در تابع GetCount که تابعی ایستا است، غیرمجاز است.

```
1  #include <iostream>
2  #include "Point.h"
3
4  using namespace std;
5
6  unsigned int Point::count = 0;
7
8  unsigned int Point::GetCount ()
9  {
10     return count;
11 }
12
13 Point::Point (int a, int b) : x(a), y(b)
14 {
15     // cout << " ("<<x<<','<<y<<") is constructed!"<<endl;
16     (this -> count)++;
17 }
18
19 Point::~~Point ()
20 {
21     // cout << " ("<<x<<','<<y<<") is destructed!"<<endl;
22     this -> count--;
23 }
24
25 void Point::display() const
26 {
27     cout << '(' << this -> x << ',' << this -> y << ')' << endl;
28 }
29
```



مثال ۱: اشاره گر this (فایل main.cpp)

```
1  #include <iostream>
2  #include "Point.h"
3
4  using namespace std;
5
6  int main()
7  {
8      Point p;
9      p.display();
10     p.Setx(100);
11     p.Sety(200);
12     p.display();
13     return 0;
14 }
15
```



```
"C:\Users\Maryam\courses\Advanced programing\ThisPoin...
(0,0)
(100,200)
Process returned 0 (0x0)   execution time : 0.619 s
Press any key to continue.
```



مثال ۲: اشاره گر this (فایل Point.h)

```
1  #ifndef POINT_H
2  #define POINT_H
3
4
5  class Point
6  {
7      friend void func(Point&);
8      public:
9          Point(int = 0, int = 0);
10         ~Point();
11
12         int Getx() const { return x; }
13         Point& Setx(int);
14         int Gety() const { return y; }
15         Point& Sety(int);
16         void display() const;
17         static unsigned int GetCount();
18
19     protected:
20
21     private:
22         int x;
23         int y;
24         static unsigned int count;
25 };
26
27 #endif // POINT_H
```

خروجی توابع Setx و Sety از void به Point& تغییر کرده است.



مثال ۲: اشاره گر this (فایل Point.cpp)

```
1  #include <iostream>
2  #include "Point.h"
3
4  using namespace std;
5
6  unsigned int Point::count = 0;
7
8  unsigned int Point::GetCount()
9  {
10     return count;
11 }
12
13 Point::Point(int a, int b) : x(a), y(b)
14 {
15     // cout << " ("<<x<<','<<y<<" is constructed!"<<endl;
16     count++;
17 }
18
19 Point::~~Point()
20 {
21     // cout << " ("<<x<<','<<y<<" is destructed!"<<endl;
22     count--;
23 }
24
25 Point& Point::Setx(int val)
26 {
27     x = val;
28     return *this;
29 }
30
31 Point& Point::Sety(int val)
32 {
33     y = val;
34     return *this;
35 }
36
37 void Point::display() const
38 {
39     cout << '(' << this -> x << ',' << this -> y << ')' << endl;
40 }
41
```

توابع Setx و Sety خود شیء را برگردانده‌اند، برای این کار از اشاره گر **this** استفاده شده است.



مثال ۲: اشاره گر this (فایل main.cpp)

```
1  #include <iostream>
2  #include "Point.h"
3
4  using namespace std;
5
6  int main()
7  {
8      Point p;
9      p.display();
10     p.Setx(100).Sety(200).display();
11
12     return 0;
13 }
14
```

با توجه به تغییرات انجام شده در توابع Setx و Sety، امکان فراخوانیهای متوالی با استفاده نام شی فراهم شده است.

"C:\Users\Maryam\courses\Advanced programming\ThisPoin..."

(0,0)
(100,200)

Process returned 0 (0x0) execution time : 0.110 s
Press any key to continue.



بازتعریف عملگرها (Operator Overloading)

- در C++ مجموعه متنوعی از عملگرها وجود دارد که برای اهداف خاصی برنامه‌ریزی شده‌اند. C++ این امکان را فراهم آورده است که برنامه‌نویسان این عملگرها را برای اهداف خود بازتعریف کنند.
- بعضی عملگرها در C++ هم برای عملیات مختلف مورد استفاده قرار گرفته‌اند. (مثل >> که هم به عنوان عملگر استخراج و هم به عنوان عملگر بیتی جابجایی استفاده می‌شود و یا عملگر + که برای جمع اعداد صحیح، اعشاری، رشته‌ها و ... استفاده می‌شود). مشابه استفاده از توابع همنام (function overloading)، کامپایلرها با توجه به نوع داده عملوندها عملیات مناسب را انتخاب می‌کند.
- در C++ اگرچه نمی‌توان عملگرهای جدید ایجاد کرد؛ اما این امکان وجود دارد که اکثر عملگرها بازتعریف شوند تا عملیات موردنظر برنامه‌نویسان برای کلاسهای ایجاد شده پیاده‌سازی شود.
- عملگرها را می‌توان با استفاده از توابع مستقل یا توابع غیرایستای عضو کلاس بازتعریف کرد.
- در هنگام بازتعریف عملگرها نام تابع ترکیب کلمه کلیدی **operator** و **نماد عملگر** خواهد بود. مثلاً برای بازتعریف عملگر + باید نام تابع **operator+** باشد.
- عملوندها به عنوان پارامترهای ورودی تابع در نظر گرفته می‌شود. در صورتی که تابع به صورت تابع عضو کلاس پیاده‌سازی شود، حتماً باید به صورت تابع غیر ایستا ایجاد شود. چون با شئی از کلاس فراخوانی می‌شود و روی داده‌های آن شئی عمل می‌کند.



بازتعریف عملگرها (Operator Overloading)

• جدول زیر تعدادی از عملگرهای C++ که می‌توان بازتعریف کرد، را نشان می‌دهد:

+	-	*	/	%	^
&		~	!	,	=
<	>	<=	>=	++	--
<<	>>	==	!=	&&	
+=	-=	/=	%=	^=	&=
=	*=	<<=	>>=	[]	()
->	->*	new	new []	delete	delete []

• عملگرهای زیر را نمی‌توان بازتعریف کرد:

:: .* . ?:



بازتعریف عملگرها (Operator Overloading)

- در بازتعریف عملگرها قوانین و محدودیتهایی وجود دارد که باید آنها را در نظر داشت:
- اولویت عملگرها با بازتعریف آنها تغییر نمی‌کند و در صورت نیاز می‌توان با استفاده از پرانتزها ترتیب دلخواه را در عبارات به وجود آورد.
- با بازتعریف عملگرها، ترتیب انجام عملگرهای هم اولویت نیز تغییر نخواهد کرد.
- با بازتعریف عملگرها، چندتایی عملگر هم تغییر نخواهد کرد. یعنی اگر عملگری باینری است، باید باینری بازتعریف شود و اگر یکانی است به صورت یکانی. البته بعضی از عملگرها هم باینری و هم یکانی (مثل +، -، *، &) هستند، هر دو شکل این عملگرها به صورت جداگانه می‌تواند بازتعریف شوند.
- نمی‌توان عملگرها را برای انواع داده پایه بازتعریف کرد. فقط می‌توان عملگرها را برای انواع داده تعریف شده توسط برنامه‌نویس (اشیا کلاسه‌های جدید) یا ترکیبی از انواع داده تعریف شده توسط برنامه‌نویس و انواع داده پایه بازتعریف کرد.
- عملگرهای مرتبط مثل + و += را باید به صورت جداگانه بازتعریف کرد.
- عملگرهای ()، []، > و هر گونه عملگر انتساب باید به صورت تابع عضو کلاس پیاده‌سازی شود.
- عملگرهای =، & و عملگر کاما از قبل برای انواع داده تعریف شده توسط برنامه‌نویس وجود دارد، ولی بقیه عملگرها حتما باید برای انواع داده تعریف شده توسط برنامه‌نویس بازتعریف شوند تا بتوان از آنها استفاده کرد. خود عملگرهای ذکر شده را نیز می‌توان در صورت نیاز بازتعریف کرد.



مثال ۳: بازتعریف عملگرها (فایل Point.h)

```
1  #ifndef POINT_H
2  #define POINT_H
3
4  #include<iostream>
5
6  using namespace std;
7
8  class Point
9  {
10     friend ostream& operator<<(ostream&, const Point&);
11     friend istream& operator>>(istream&, Point&);
12     friend bool operator==(const Point&, const Point&);
13     friend Point operator+(const Point&, const Point&);
14     friend Point& operator++(Point&); // Prefix increment
15     friend Point operator++(Point&, int); // Postfix increment
16 public:
17     Point(int = 0, int = 0);
18     ~Point();
19
20     int GetX() const { return x; }
21     void SetX(int val) { x = val; }
22     int GetY() const { return y; }
23     void SetY(int val) { y = val; }
24     void display() const;
25     Point operator-(const Point&) const;
26     Point& operator--(); // Prefix decrement
27     Point operator--(int); //Postfix decrement
28     int operator[] (int);
29     static unsigned int GetCount();
30
31 protected:
32
33 private:
34     int x;
35     int y;
36     static unsigned int count;
37 };
38
39 #endif // POINT_H
```

بازتعریف عملگرها به صورت تابع مستقل (غیرعضو) دوست

بازتعریف عملگرها به صورت تابع عضو کلاس



مثال ۳: بازتعریف عملگرها (فایل Point.cpp)

```
1  #include <iostream>
2  #include "Point.h"
3
4  using namespace std;
5
6  unsigned int Point::count = 0;
7
8  ostream& operator<<(ostream& out, const Point& p)
9  {
10     out << '(' << p.x << ',' << p.y << ')' << endl;
11     return out;
12 }
13 istream& operator>>(istream& in, Point& p)
14 {
15     cout << " Enter x:";
16     in >> p.x;
17     cout << " Enter y:";
18     in >> p.y;
19     return in;
20 }
21 bool operator==(const Point &p1, const Point &p2)
22 {
23     if(p1.x == p2.x && p1.y == p2.y)
24         return true;
25     return false;
26 }
27 Point operator +(const Point& p1, const Point& p2)
28 {
29     Point sum;
30     sum.x = p1.x + p2.x;
31     sum.y = p1.y + p2.y;
32     return sum;
33 }
34 Point& operator ++(Point &p)
35 {
36     p.x++;
37     p.y++;
38     return p;
39 }
40 Point operator++(Point& p, int)
41 {
42     Point temp(p);
43     p.x++;
44     p.y++;
45     return temp;
46 }
```

بازتعریف عملگر << به صورت تابع مستقل (غیرعضو) دوست ←

بازتعریف عملگر >> به صورت تابع مستقل (غیرعضو) دوست ←

بازتعریف عملگر == به صورت تابع مستقل (غیرعضو) دوست ←

بازتعریف عملگر + به صورت تابع مستقل (غیرعضو) دوست ←

بازتعریف عملگر ++ پیشوندی به صورت تابع مستقل (غیرعضو) دوست ←

بازتعریف عملگر ++ پسوندی به صورت تابع مستقل (غیرعضو) دوست ←



مثال ۳: بازتعریف عملگرها (ادامه فایل Point.cpp)

```
47 unsigned int Point::GetCount()
48 {
49     return count;
50 }
51 Point::Point(int a, int b) : x(a), y(b)
52 {
53     //cout << " ("<<x<<','<<y<<" is constructed!"<<endl;
54     count++;
55 }
56 Point::~~Point()
57 {
58     //cout << " ("<<x<<','<<y<<" is destructed!"<<endl;
59     count--;
60 }
61 void Point::display() const
62 {
63     cout << '(' << x << ',' << y << ')' << endl;
64 }
65 Point Point::operator -(const Point& p) const
66 {
67     Point result;
68     result.x = x - p.x;
69     result.y = y - p.y;
70     return result;
71 }
72 Point& Point::operator --()
73 {
74     x--;
75     y--;
76     return *this;
77 }
78 Point Point::operator--(int)
79 {
80     Point temp;
81     temp = *this;
82     x--;
83     y--;
84     return temp;
85 }
86 int Point::operator[](int index)
87 {
88     if (index%2 == 0)
89         return x;
90     return y;
91 }
```

← بازتعریف عملگر - به صورت تابع عضو کلاس

← بازتعریف عملگر -- پیشوندی به صورت تابع عضو کلاس

← بازتعریف عملگر -- پسوندی به صورت تابع عضو کلاس

← بازتعریف عملگر [] پیشوندی به صورت تابع عضو کلاس



مثال ۳: بازتعریف عملگرها (فایل Circle.h)

```
1  #ifndef CIRCLE_H
2  #define CIRCLE_H
3
4  #include "Point.h"
5
6  class Circle
7  {
8      friend ostream& operator <<(ostream& out, const Circle& c);
9      friend istream& operator >>(istream& in, Circle& c);
10 public:
11     Circle(int = 1, int = 1, int = 1);
12     Circle(Point &, int = 1);
13     ~Circle();
14     void SetRadius(int r);
15     int GetRadius() const;
16     void SetCenter(Point &);
17     Point GetCenter() const;
18     float GetPrimeter() const;
19     float GetArea() const;
20     Circle operator+(const Circle&) const;
21     Circle operator+(const int&) const;
22     Circle operator+=(const int&);
23     bool operator==(const Circle&) const;
24     bool operator<(const Circle&) const;
25     bool operator>(const Circle&) const;
26     static double PI;
27
28
29 private:
30     int Radius;
31     Point Center;
32 };
33
34 #endif // CIRCLE_H
```

بازتعریف عملگرها به صورت تابع مستقل (غیرعضو) دوست

بازتعریف عملگرها به صورت تابع عضو کلاس



مثال ۳: بازتعریف عملگرها (فایل Circle.cpp)

```
1  #include "Circle.h"
2  #include <iostream>
3  using namespace std;
4
5  double Circle::PI = 3.14;
6
7  ostream &operator <<(ostream &out, const Circle &c)
8  {
9      out << "radius:" << c.Radius << '\t';
10     c.Center.display();
11     return out;
12 }
13
14 istream &operator >>(istream &in, Circle &c)
15 {
16     int x, y;
17     cout << " Enter radius: ";
18     in >> c.Radius;
19     cout << " Enter Center:\n" ;
20     in >> c.Center;
21     return in;
22 }
23
24 Circle::Circle(int x, int y, int r) : Center(x, y)
25 {
26     SetRadius(r);
27     //cout << "circle was constructed! Radius = " << Radius << ",Center = ";
28     //Center.display();
29 }
30
31 Circle::Circle(Point &p, int r) : Center(p)
32 {
33     SetRadius(r);
34     //cout << "circle was constructed! Radius = " << Radius << ",Center = ";
35     //Center.display();
36 }
37
38 Circle::~Circle()
39 {
40     // cout << "circle was desstructured! Radius = " << Radius << ",Center = ";
41     // Center.display();
42 }
43
44
```

بازتعریف عملگر << به صورت تابع مستقل (غیرعضو) دوست ←

بازتعریف عملگر >> به صورت تابع مستقل (غیرعضو) دوست ←



مثال ۳: بازتعریف عملگرها (ادامه فایل Circle.cpp)

```
45 void Circle::SetCenter(Point &p)
46 {
47     Center.SetX(p.GetX());
48     Center.SetY(p.GetY());
49     // Center = p;
50 }
51
52 Point Circle::GetCenter() const
53 {
54     return Center;
55 }
56
57 void Circle::SetRadius(int r)
58 {
59     if (r > 0)
60         Radius = r;
61     else
62     {
63         cout << "Radius is invalid! Set Radius = 1" << endl;
64         Radius = 1;
65     }
66 }
67
68 int Circle::GetRadius() const
69 {
70     return Radius;
71 }
72
73 float Circle::GetPrimeter() const
74 {
75     return 2 * Radius * PI;
76 }
77
78 float Circle::GetArea() const
79 {
80     return Radius * Radius * PI;
81 }
```



مثال ۳: بازتعریف عملگرها (ادامه فایل Circle.cpp)

```
82 Circle Circle::operator+(const Circle& c) const
83 {
84     Circle sum;
85     sum.Center = Center + c.Center;
86     sum.Radius = Radius + c.Radius;
87     return sum;
88 }
```

← بازتعریف عملگر + به صورت تابع عضو کلاس

```
91 Circle Circle::operator+(const int &a) const
92 {
93     Circle sum(*this);
94     sum.Radius = Radius + a;
95     return sum;
96 }
```

← بازتعریف عملگر + به صورت تابع عضو کلاس

```
98 Circle Circle::operator+=(const int &a)
99 {
100     Radius += a;
101     return *this;
102 }
```

← بازتعریف عملگر += به صورت تابع عضو کلاس

```
104 bool Circle::operator<(const Circle& c) const
105 {
106     if(Radius < c.Radius)
107         return true;
108     return false;
109 }
```

← بازتعریف عملگر < پیشوندی به صورت تابع عضو کلاس

```
111 bool Circle::operator>(const Circle& c) const
112 {
113     if(Radius > c.Radius)
114         return true;
115     return false;
116 }
```

← بازتعریف عملگر > پیشوندی به صورت تابع عضو کلاس

```
118 bool Circle::operator==(const Circle& c) const
119 {
120     if(Radius == c.Radius && Center == c.Center)
121         return true;
122     return false;
123 }
```

← بازتعریف عملگر == پیشوندی به صورت تابع عضو کلاس



مثال ۳: بازتعریف عملگرها (فایل main.cpp)

```
1  #include <iostream>
2  #include "Point.h"
3  #include "Circle.h"
4
5  using namespace std;
6
7  int main()
8  {
9      cout << "\nMAIN FUNCTION: EXECUTION BEGINS" << endl;
10
11     Point point1(10,10), point2, point3;
12     Circle circle1(point1, 10), circle2, circle3;
13     cout << "Point1:" << point1 << "point2:" << point2 << "Point3:" << point3;
14     cout << "circle1 " << circle1 << "circle2 " << circle2 << "circle3 " << circle3;
15     cout << "point2:\n";
16     cin >> point2;
17     cout << "circle2:\n";
18     cin >> circle2;
19     point3 = point1 + point2;
20     circle1 += 100;
21     circle2 = circle2 + 10;
22     //circle2 = 10 + circle1; Error
23     circle3 = circle1 + circle2;
24     cout << "Point1:" << point1 << "point2:" << point2 << "Point3:" << point3;
25     cout << "circle1 " << circle1 << "circle2 " << circle2 << "circle3 " << circle3;
26     point3 = point2 - point1;
27     cout << "Point1++:" << point1++ << "++point2:" << ++point2 << "Point3:" << point3;
28     cout << "Point1--:" << point1-- << "--point2:" << --point2 << "Point3:" << point3;
29     cout << "point1[0] = " << point1[0] << ", point1[1] = " << point1[1] << endl;
30     cout << "point2[4] = " << point2[4] << ", point2[7] = " << point2[7] << endl;
31     if (circle1 > circle2)
32         cout << "circle1 is greater than circle2!" << endl;
33     else if (circle1 < circle2)
34         cout << "circle1 is smaller than circle2!" << endl;
35     else
36         cout << "circle1 is equal to circle2!" << endl;
37
38     cout << "\nMAIN FUNCTION: EXECUTION ENDS" << endl;
39     return 0;
40 }
```

```
"C:\Users\Maryam\courses\Advanced programming\OverLoa...
MAIN FUNCTION: EXECUTION BEGINS
Point1:(10,10)
point2:(0,0)
Point3:(0,0)
circle1 radius:10      (10,10)
circle2 radius:1      (1,1)
circle3 radius:1      (1,1)
point2:
Enter x:100
Enter y:100
circle2:
Enter radius: 5
Enter Center:
Enter x:50
Enter y:50
Point1:(10,10)
point2:(100,100)
Point3:(110,110)
circle1 radius:110     (10,10)
circle2 radius:15     (50,50)
circle3 radius:125     (60,60)
Point1++:(10,10)
++point2:(101,101)
Point3:(90,90)
Point1--:(11,11)
--point2:(100,100)
Point3:(90,90)
point1[0] = 10, point1[1] = 10
point2[4] = 100, point2[7] = 100
circle1 is greater than circle2!

MAIN FUNCTION: EXECUTION ENDS

Process returned 0 (0x0)   execution time : 32.620 s
Press any key to continue.
```

بازتعریف عملگرها

- در بازتعریف عملگرها به صورت توابع عضو کلاس نیازی به ارسال داده‌های کلاس به تابع وجود ندارد؛ بنابراین پارامترهای ورودی در این حالت در مقایسه با بازتعریف به شکل توابع مستقل (توابع غیرعضو) کمتر است.
- در بازتعریف عملگرهای دودویی فراخوانی برای شی عملوند اول صورت می‌گیرد؛ به همین علت اگر شی سمت چپ مربوط به کلاس تعریف شده برنامه‌نویس نباشد، نمی‌توان عملگر را به شکل تابع عضو بازتعریف کرد. مثل عملگر درج و استخراج
- عملگرهای افزایش (++) و کاهش (--) به دو صورت پیشوندی و پسوندی می‌توانند بازتعریف شوند؛ اما چون باید امضای دو تابع متفاوت باشد، در شکل پسوندی یک پارامتر صحیح به پارامترهای ورودی اضافه شده است که صرفاً برای تمایز در امضای دو تابع و تشخیص کامپایلر برای فراخوانی تابع است و از این مقدار در پیاده‌سازی تابع استفاده‌ای نمی‌شود.
- در بازتعریف عملگرهای به صورت توابع مستقل در صورت عدم دسترسی و استفاده از متغیرهای خصوصی نیازی به دوست (friend) معرفی کردن تابع نیست.
- ترتیب عملوندها در بازتعریف اهمیت زیادی دارد. برای مثال اگر در بازتعریف عملگری عملوند اول از نوع کلاس ایجاد شده توسط برنامه‌نویس و عملوند دوم از نوع داده صحیح تعریف شده باشد، نمی‌توان از این بازتعریف برای حالتی که عملوند اول از نوع داده صحیح و عملوند دوم از نوع کلاس ایجاد شده توسط برنامه‌نویس استفاده کرد و در صورت نیاز باید آن حالت را نیز به صورت جداگانه بازتعریف کرد.



مرور مطالب

• مثال ۴: برای کلاس Date عملگرهای زیر را بازتعریف (overload) کنید:

- عملگرهای < و > را برای دریافت و چاپ تاریخ باز تعریف کنید.
- عملگرهای ++ پیشوندی و پسوندی را برای محاسبه تاریخ روز بعد بازتعریف کنید.
- عملگرهای -- پیشوندی و پسوندی را برای محاسبه تاریخ روز قبل بازتعریف کنید.
- عملگر += را به گونه‌ای بازتعریف کنید که با دریافت یک عدد صحیح مانند X به عنوان عملوند دوم، تاریخ X روز بعد را محاسبه کند.
- عملگر < را به گونه‌ای بازتعریف کنید که اگر تاریخ اول (عملوند اول) قبل از تاریخ دوم (عملوند دوم) است مقدار درست و در غیر این صورت مقدار نادرست را برگرداند.
- عملگر > را به گونه‌ای بازتعریف کنید که اگر تاریخ اول (عملوند اول) بعد از تاریخ دوم (عملوند دوم) است مقدار درست و در غیر این صورت مقدار نادرست را برگرداند.
- عملگر == را به گونه‌ای بازتعریف کنید که اگر دو تاریخ برابر است مقدار درست و در غیر این صورت مقدار نادرست را برگرداند.



(مثال ۴: فایل Date.h)

```
1  #ifndef DATE_H
2  #define DATE_H
3
4  #include<iostream>
5
6  using namespace std;
7
8  class Date
9  {
10 public:
11
12     Date(int =1,int =1,int =1402);
13     ~Date(){};
14     void setDate(int, int, int);
15     int getDay() const {return day;}
16     int getMonth() const {return month;}
17     int getYear() const {return year;}
18     void print() const;
19     Date operator ++(int);
20     Date operator ++();
21     Date operator --(int);
22     Date operator --();
23     Date operator +=(int);
24     bool operator ==(const Date &) const;
25     bool operator <(const Date &) const;
26     bool operator >(const Date &) const;
27     static int getDaysOfMonth(int m) {return DaysOfMonth[m];}
28
29 private:
30     int day;
31     int month;
32     int year;
33     static int DaysOfMonth[];
34     bool kabise() {return false;} // if the year is a leap year, return true; otherwise, return false
35     void setDay(int);
36     void setMonth(int);
37     void setYear(int);
38     void nextDay();
39     void previousDay();
40 };
41 ostream &operator <<(ostream &out, const Date&);
42 istream &operator >>(istream &, Date &);
43
44 #endif // DATE_H
```

نیازی به **friend** معرفی کردن این توابع نیست زیرا این تابع تنها از توابع عمومی کلاس استفاده کرده است.
در اینجا تنها **prototype** تابع برای استفاده ذکر شده است.



(مثال ۴: فایل Date.cpp)

```
1  #include <iostream>
2  #include "Date.h"
3
4  using namespace std;
5
6  int Date::DaysOfMonth[]={0,31,31,31,31,31,31,31,30,30,30,30,29};
7
8  Date::Date(int d,int m,int y)
9  {
10     setDate(d, m, y);
11 }
12 void Date::setYear(int y)
13 {
14     if(y>0)
15         year = y;
16     else
17     {
18         cout<<"year is invalid! set year=1402."<<endl;
19         year = 1402;
20     }
21 }
22
23 void Date::setMonth(int m)
24 {
25     if(m>=1 && m<=12)
26         month = m;
27     else
28     {
29         cout<<"month is invalid! Set month =1"<<endl;
30         month = 1;
31     }
32 }
33
34 void Date::setDay(int d)
35 {
36     if(d >= 1 && d <= DaysOfMonth[month])
37         day = d;
38     else
39     {
40         if (d == 30 && month ==12 && kabise())
41             day = d;
42         else
43         {
44             cout<<"day is invalid! set day = 1."<<endl;
45             day = 1;
46         }
47     }
48 }
```



مثال ۴: (ادامه فایل Date.cpp)

```
47 void Date::nextDay()
48 {
49     if (day < DaysOfMonth[month])
50         day++;
51     else
52         if (month == 12)
53         {
54             if (kabise() && day == 29)
55                 day++;
56             else
57             {
58                 year++;
59                 month = 1;
60                 day = 1;
61             }
62         }
63     else
64     {
65         month++;
66         day = 1;
67     }
68 }
69 void Date::previousDay()
70 {
71     if (day > 1)
72         day--;
73     else
74         if (month == 1)
75         {
76             year--;
77             month = 12;
78             if (kabise())
79                 day = 30;
80             else
81                 day = 29;
82         }
83     else
84     {
85         month--;
86         day = DaysOfMonth[month];
87     }
88 }
```



مثال ۴: (ادامه فایل Date.cpp)

```
89 Date Date::operator++(int)
90 {
91     Date temp;
92     temp = *this;
93     nextDay();
94     return temp;
95 }
96
97 Date Date::operator++()
98 {
99     nextDay();
100    return *this;
101 }
102
103 Date Date::operator--(int)
104 {
105     Date temp;
106     temp = *this;
107     previousDay();
108     return temp;
109 }
110
111 Date Date::operator--()
112 {
113     previousDay();
114     return *this;
115 }
116
117 Date Date::operator+=(int x)
118 {
119     for(int i=0; i<x; i++)
120         nextDay();
121     return *this;
122 }
123 bool Date::operator==(const Date &d) const
124 {
125     if(d.day == day && d.month == month && d.year == year)
126         return true;
127     return false;
128 }
```



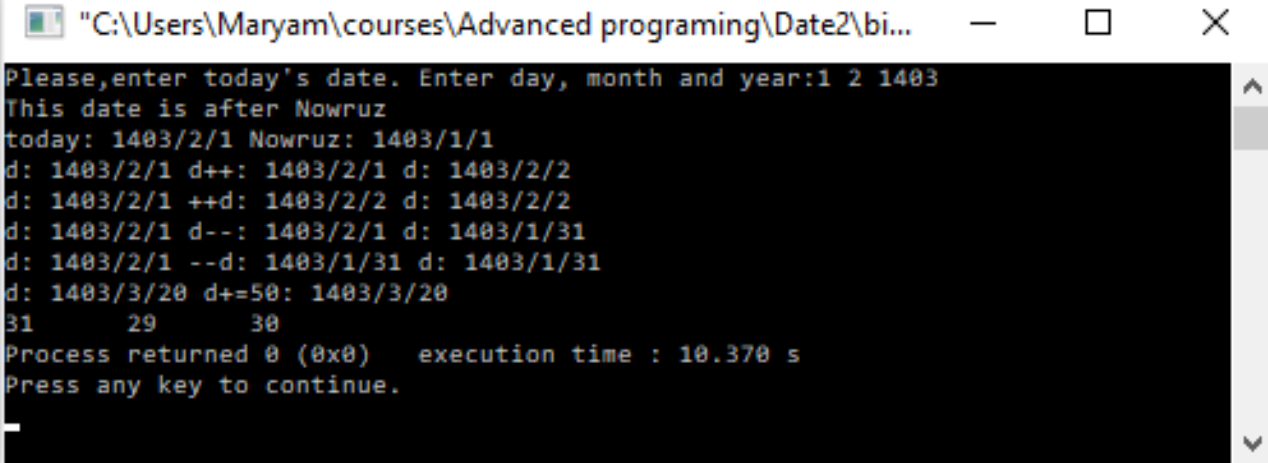
مثال ۴: (ادامه فایل Date.cpp)

```
129 bool Date::operator<(const Date &d) const
130 {
131     if (year < d.year)
132         return true;
133     if (year == d.year && month < d.month)
134         return true;
135     if (year == d.year && month == d.month && day < d.day)
136         return true;
137     return false;
138 }
139 bool Date::operator>(const Date &d) const
140 {
141     if (year > d.year)
142         return true;
143     if (year == d.year && month > d.month)
144         return true;
145     if (year == d.year && month == d.month && day > d.day)
146         return true;
147     return false;
148 }
149 void Date::setDate(int d, int m, int y)
150 {
151     setYear(y);
152     setMonth(m);
153     setDay(d);
154 }
155 void Date::print() const
156 {
157     cout<<year<<"/"<<month<<"/"<<day;
158 }
159
160 istream &operator >>(istream &in, Date &date)
161 {
162     int d,m,y;
163     cout <<"Enter day, month and year:";
164     in >> d >> m >> y;
165     date.setDate(d, m, y);
166     return in;
167 }
168 ostream &operator <<(ostream &out, const Date &date)
169 {
170     date.print();
171     return out;
172 }
```



مثال ۴: (فایل main.cpp)

```
1  #include <iostream>
2  #include "Date.h"
3
4  using namespace std;
5
6  int main()
7  {
8      Date today,d;
9      const Date Nowruz(1,1,1403);
10
11     cout << "Please,enter today's date.";
12     cin >> today;
13
14     if(Nowruz == today)
15         cout<<"Happy New year!" << endl;
16     if(Nowruz < today)
17         cout << "This date is after Nowruz" << endl;
18     if(Nowruz > today)
19         cout << "This date is before Nowruz" << endl;
20
21     cout << "today: " << today << " Nowruz: " << Nowruz << endl;
22     d = today;
23     cout << "d: " << d << " d++: " << d++ << " d: " << d << endl;
24     d = today;
25     cout << "d: " << d << " ++d: " << ++d << " d: " << d << endl;
26     d = today;
27     cout << "d: " << d << " d--: " << d-- << " d: " << d << endl;
28     d = today;
29     cout << "d: " << d << " --d: " << --d << " d: " << d << endl;
30     d = today;
31     d+=50;
32     cout << "d: " << d << "d+=50: " << d << endl;
33
34     cout << Date::getDaysOfMonth(5) <<'\t';
35     cout << today.getDaysOfMonth(12) <<'\t';
36     cout << Nowruz.getDaysOfMonth(7) <<'\t';
37
38     return 0;
39 }
```



```
"C:\Users\Maryam\courses\Advanced programing\Date2\bi...
Please,enter today's date. Enter day, month and year:1 2 1403
This date is after Nowruz
today: 1403/2/1 Nowruz: 1403/1/1
d: 1403/2/1 d++: 1403/2/1 d: 1403/2/2
d: 1403/2/1 ++d: 1403/2/2 d: 1403/2/2
d: 1403/2/1 d--: 1403/2/1 d: 1403/1/31
d: 1403/2/1 --d: 1403/1/31 d: 1403/1/31
d: 1403/3/20 d+=50: 1403/3/20
31      29      30
Process returned 0 (0x0)   execution time : 10.370 s
Press any key to continue.
```

